Programmation C-Avancé

TP sur les tables de hachage

Durée 3h

Table des matières

1. Introduction : 3
2. Arborescence du projet 3
3. Compilation, ... 6
4. Table de hachage 7
5. A faire 8
6. Fonctions sur les listes chaînées 10

1. Introduction :

On se propose dans ce premier TP de coder la gestion de table de hachage. Il est demandé ici de compléter les fonctions du fichier hash.c vue en cours avec les tests correspondants à chaque fonction. Les fonctions de gestion des listes chaînées ainsi que les Cmake pour la construction du projet global sont fournis.

2. Arborescence du projet

```linux
1       hash_tp
2           debug
3                hash
4                list
5           src
6                 hash
```

```
7                       CMakeLists.txt
8                       HachConfig.cmake.in
9                       HashConfig.cmake.in
10                      hash.c
11                      hash.h
12                      hash.inc
13                      test-hash.c
14                  list
15                      CMakeLists.txt
16                      ListConfig.cmake
17                      ListConfig.cmake.in
18                      list.c
19                      list.h
```

```
20                      list.pc.in
21                      test-list.c
22                tmp
23                       include
24                       lib
25
```

3. Compilation,

a) Positionnez vous dans dans un premier temps le repertoire debug/list

b) lancez les commandes suivantes

```linux
- cmake ../../src/list/ -DCMAKE_INSTALL_PREFIX=../tmp -DCMAKE_BUILD_TYPE=Debug
- make
- make install
```

pour l'installation de la librairie liste afin qu'elle puisse être utilisée par d'autres programmes

Si aucune erreur (c'est le cas avec ce qui vous est donné).

c)refaire la même chose en vous positionnant dans le répertoire Debug / Hash

4. Table de hachage

Pour la table de hashage on utilisera la structure suivantes (vue en cours):

```
1 typedef struct {                                                    Vim
2     size_t              (*hash) (const void *, size_t);
3     int                 (*compare) (const void *, const void *);
4     size_t              length;
5     size_t              nb;
6     size_t              size;
7     List                list[1];
8 }HashTable;
```

5. A faire

Complétez l écriture des fonctions où il est ecrit « A completer » et testez les sur un programme à créer test-hash.c

Certaines fonctions sont déjà fournies.

```
1  HashTable*                                                    Vim
2  hashtable_new(size_t size, size_t(*hash) (const void *, size_t),
3  int (*compare) (const void *, const void *));
4
5  void
6  hashtable_apply(HashTable * table, void (*func) (void *, void *), void
   *extra_data);
7  void
```

```
8
9   hashtable_print(HashTable * table, void (*print) (void *, FILE *),
    FILE * stream);
10  void
11
12  hashtable_delete(HashTable * table, void (*delete) (void *));
13  int
14
15  hashtable_insert(HashTable ** ptable, void *data, void (*delete) (void
    *));
16
17  void                *
18  hashtable_lookup(HashTable * table, void *data);
```

```
19
20  int
21  hashtable_retract(HashTable ** ptable, void *data, void (*delete)
    (void *));
```

6. Fonctions sur les listes chaînées

Les fonctions de manipulation des listes chaînées sont fournies et sont les suivantes.

Vous n utiliserez bien sûr que celles dont vous aurez besoin pour la table de hachage.

```
1     // function creating a list                                  Vim
2    List list_new (void);
3
4    /* function removing the first element of a list.
```

```
5       TRUE is returned in the case of a success, FALSE otherwise */
6
7    int list_remove_first (List * plist, void (*delete) (void *));
8
9    /*
10    * function removing the last element of a list.
11    * TRUE is returned in the case of a success, FALSE otherwise
12    */
13   int list_remove_last (List * plist, void (*delete) (void *));
14
15   /*
16    * function removing the nth element of a list.
17    * TRUE is returned in the case of a success, FALSE otherwise
```

```
 */
int list_remove_nth (List * plist, size_t nth, void (*delete) (void *));

/*
 * function deleting a list entirely
 */
void list_delete (List * plist, void (*delete) (void *));

/*
 * function computing the length of a list
 */
size_t
```

```
30  list_length (List list);
31
32  /*
33   * function returning the test: is list empty?
34   */
35  int list_is_empty (List list);
36
37  /*
38   * function prepending an element to a list.
39   * TRUE is returned in the case of a success, FALSE otherwise
40   */
41  int list_prepend (List * plist, const void *data, size_t size);
42
```

```
43  /*
44   * function appending an element to a list
45   * TRUE is returned in the case of a success, FALSE otherwise
46   */
47  int list_append (List * plist, const void *data, size_t size);
48
49  /*
50   * function inserting an element in the nth position.
51   * TRUE is returned in the case of a success, FALSE otherwise
52   */
53  int list_insert (List * plist,
54       const void *data,
55       size_t size,
```

```
56          size_t nth);
57
58  /*
59   * function inserting an element in a sorted list
60   * the compare function has the same meaning in the standard qsort
     function
61   */
62  int list_insert_sorted (List * plist,
63          const void *data,
64          size_t size,
65          int (*compare) (const void *, const void *));
66
67  /*
```

```
68    * function to give the pointer to the first data
69    * NULL is returned if the list is empty
70    */
71   void *list_first_data (List list);
72
73   /*
74    * function giving the pointer to the last data
75    * NULL is returned if the list is empty
76    */
77   void *list_last_data (List list);
78
79   /*
80    * function giving the pointer to the nth data
```

```
81    * NULL is returned if the case of an error
82    */
83   void *list_nth_data (List list, size_t nth);
84
85
86   /*
87    * function giving the last sublist of a list.
88    * NULL is returned if the list is empty
89    */
90   List
91   list_last (List list);
92
93   /*
```

```
94    * function giving the nth sublist of a list.
95    * NULL is returned if the case of an error
96    */
97   List
98   list_nth (List list, size_t nth);
99
100  /*
101   * function finding the pointer to a data.
102   * NULL is returned if the case of an error: the data does not exist
103   * in the list
104   */
105  void *list_find_data (List list,
106            const void *data,
```

```
107             int (*compare) (const void *, const void *));
108
109 /*
110  * function giving the index of a data. -1 is returned if the data
111  * is not in the list
112  */
113 int list_index_data (List list,
114             const void *data,
115             int (*compare) (const void *, const void *));
116
117 /*
118  * function applicating an operation to the entire list
```

```
119  * extra_data is passed as the second argument of the func function
     for each
120  * element of the list
121  */
122 void list_foreach (List list, void (*func) (void *, void *), void
     *extra_data);
123
124
125 /*
126  * function concatening the list add to the list pointed by plist
127  */
128 void list_concat (List * plist, List add);
129
```

```
130 /*
131  * function splitting a list into two list
132  * *pbefore will contain all elements "inferior or equal" to data
133  * *pafter will contain all elements "superior" to data
134  */
135 void list_split (List * plist,
136      List * pbefore,
137      List * pafter,
138      const void *data,
139      int (*compare) (const void *, const void *));
140
141 /*
142  * function sorting a list
```

```
143  */
144 void list_sort (List * plist, int (*compare) (const void *, const
    void *));
145
146 /*
147  * function merging two list. The list *plist will be sorted
     according
148  * to the function compare
149  */
150 void list_merge (List * plist,
151      List add,
152      int (*compare) (const void *, const void *));
```